Hochschule Karlsruhe
University of
Applied Sciences

Rusts Konzept der Speichersicherheit

Seminararbeit
von

David Grandel

Fakultät für Informatik und Wirtschaftsinformatik

Betreuer: Prof. Dr. Carsten Sinz
Bearbeitungszeitraum: Sommersemester 2026

Zusammenfassung

Deutsche Version.

Inhaltsverzeichnis

Zusammenfassung **i**

1 Einleitung **1**

2 Speichersicherheit als zentrales Sicherheitsproblem **2**

2.1 Erfassung und Bewertung von Schwachstellen . . . 3
2.2 Begriff der Speichersicherheit . . . 5
2.3 Use After Free . . . 6
2.4 Out of Bounds Write . . . 7
2.5 Sicherheitsrelevante Konsequenzen . . . 10
2.6 Lösungsansätze . . . 11

3 Rust's Approach **12**

3.1 Slide 19: Why use Rust? . . . 13
3.2 Slide 20: Ownership Rules . . . 13
3.3 Slide 21: Transfer of Ownership . . . 14
3.4 Slide 22: Transfer Methods . . . 14
3.5 Slide 23: Move Semantics . . . 14
3.6 Slide 24: Borrowing . . . 15
3.7 Slide 25: Lifetimes and Borrow Checker . . . 15
3.8 Slide 26: Prevention of Use after Free . . . 16
3.9 Slide 27: Prevention of Use after Free . . . 16
3.10 Slide 28: Compiletime Bounds Checking . . . 17

4 Fazit **18**

Literaturverzeichnis **19**

1 Einleitung

Sicherheit ist ein zentrales Thema in der Softwareentwicklung. Angreifer entwickeln immer wieder neue Methoden, um Softwaresystemen Schaden zuzufügen, unerlaubten Zugriff auf Daten von Nutzern oder Firmen zu erhalten oder Dienste lahmzulegen. Dabei verändern sich Technologien in der Softwareentwicklung ständig, wodurch fortlaufend neue Möglichkeiten für Angriffe entstehen.

Um herauszufinden, welche Probleme am häufigsten Sicherheitslücken verursachen, landet man relativ schnell bei der Common Weakness Enumeration, kurz CWE [1]. In dieser Liste werden verschiedene Schwachstellen aufgelistet und definiert, die bestimmte Angriffsmöglichkeiten von Software beschreiben.

Die größte Schwachstelle laut der aktuellsten Statistik der CWE aus dem Jahr 2025 ist tatsächlich eine Injection Schwachstelle [2], genauer gesagt die OS Command Injection. Sie entsteht, wenn Entwickler eine Anwendung nicht ausreichend gegen bösartige Argumente absichern. Um diese Schwachstelle soll es in dieser Arbeit jedoch nicht gehen.

Platz zwei und drei im Ranking beschreiben Fehler, die mit der Speichersicherheit zusammenhängen. Ein Use After Free Bug landet auf Platz zwei des Rankings. Dabei handelt es sich um einen Programmierfehler, der dazu führt, dass über einen Dangling Pointer erneut auf Speicher zugegriffen wird, der bereits zur weiteren Nutzung freigegeben wurde. Ein Out of Bounds Write steht tatsächlich auf Platz drei, während dieser Fehler in der Statistik aus dem Jahr 2024 sogar Platz eins belegte [3]. Beim Out of Bounds Write wird Speicher geschrieben, der außerhalb des gültigen Bereichs eines Puffers liegt, typischerweise auf dem Stack. Solche Fehler treten besonders häufig in Programmiersprachen mit manueller Speicherverwaltung auf, allen voran in C. Dort liegt die Verwaltung des Speichers vollständig in der Hand des Entwicklers, wodurch eine fehlerhafte Nutzung schnell zu derartigen Schwachstellen führen kann.

Diese beiden Fehlerphänomene sollen im weiteren Verlauf im Zusammenhang mit Speichersicherheit genauer betrachtet werden. Im Fokus steht das Lösungskonzept, das Rust anbietet, um derartige schwerwiegende Speicherfehler von Grund auf zu vermeiden. Die Theorie, die hinter dem Konzept der Speichersicherheit in Rust steckt, sowie kleine Beispiele dazu sollen in dieser Arbeit gezeigt werden.

2 Speichersicherheit als zentrales Sicherheitsproblem

Bevor die Speichersicherheitsprobleme im Detail betrachtet werden, lohnt sich ein Blick auf die zugrunde liegende Statistik. Dafür wird zunächst erläutert, wie die CWE Schwachstellen erfasst, bewertet und in Kategorien einteilt. So lässt sich nachvollziehen, welche Bedeutung Speichersicherheit in den Auswertungen tatsächlich einnimmt. Anschließend wird der Begriff Speichersicherheit präzisiert. Die beiden zuvor genannten Schwachstellen werden anhand kurzer C Beispiele nachvollzogen. Dabei wird Schritt für Schritt gezeigt, wie Use After Free und Out of Bounds Write in der Praxis entstehen können und warum solche Fehler in Sprachen mit manueller Speicherverwaltung besonders leicht auftreten. Zum Abschluss des Kapitels werden die sicherheitsrelevanten Konsequenzen dieser Fehler betrachtet. Darauf aufbauend werden gängige Lösungsansätze vorgestellt, um damit den Übergang zum folgenden Kapitel vorzubereiten, in dem Rusts Ansatz zur Compilezeitprüfung von Speichersicherheit im Mittelpunkt steht.

2.1 Erfassung und Bewertung von Schwachstellen

In der Einleitung wurde bereits die Common Weakness Enumeration (CWE) eingeführt, eine Sammlung, die verschiedene Softwareschwachstellen auflistet und definiert. Um vergleichbare Fehlerklassen zu erfassen, werden abstrakte Schwachstellen definiert und auf tatsächlich dokumentierte Sicherheitslücken abgebildet. Diese Zuordnung geht auf die Idee einer gemeinsamen Schwachstellenkatalogisierung zurück [4].

Die dokumentierten Einzelfälle werden als Common Vulnerabilities and Exposures (CVEs) bezeichnet. Sie stammen aus einer öffentlich zugänglichen Datenbank, in der Softwarefehler erfasst werden. Ein CVE kann eingereicht werden, wenn eine Schwachstelle beschrieben und einer CVE Nummer zugeordnet werden kann. Die Vergabe erfolgt über sogenannte CVE Numbering Authorities (CNAs), zu denen unter anderem große Technologieunternehmen wie Microsoft und Google gehören [5]. Die Datenbasis ist nicht vollständig, da nicht jeder Fehler veröffentlicht wird. Aufgrund der breiten Beteiligung gilt sie dennoch als eine der repräsentativsten frei zugänglichen Quellen für Softwareschwachstellen.

Use After Free und Out of Bounds Write aus der Einleitung teilen gemeinsame Merkmale mit zahlreichen weiteren Schwachstellen in der CWE Bewertung. Die betrachtete Auswertung bezieht sich auf Known Exploited Vulnerabilities (KEV), also nachweislich ausgenutzte Schwachstellen. Dafür werden Schwachstellen in Kategorien eingeteilt. Abbildung 1 zeigt die Verteilung der zehn am häufigsten ausgenutzten KEV Schwach-

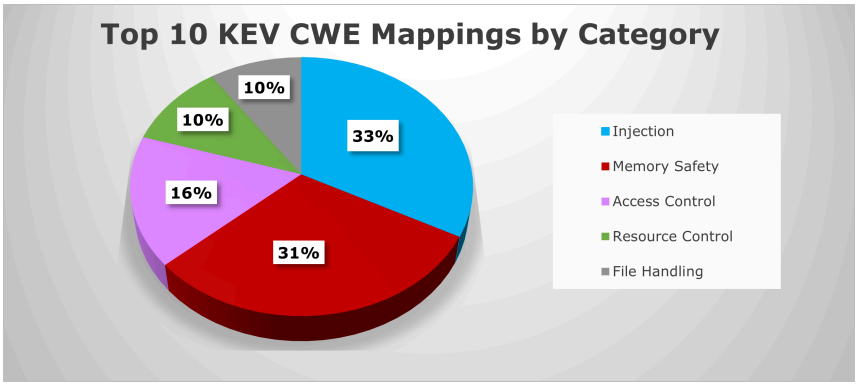


Abbildung 1 : Verteilung der CWE Top 10 KEV Schwachstellen nach Kategorie [6]

stellen nach Kategorie. Speichersicherheit (Memory Safety) nimmt mit 31 Prozent den zweitgrößten Anteil ein, direkt hinter Injection mit 33 Prozent. Weitere Kategorien wie Access Control, Resource Control und File Handling liegen deutlich zurück.

Zur Gewichtung einzelner Schwachstellen dient der sogenannte Danger Score [5]. Er kombiniert die Häufigkeit einer Schwachstelle im Datensatz mit der durchschnittlichen Schwere bei Ausnutzung. Die Häufigkeit entspricht der Anzahl der CVE Einträge, denen die Schwachstelle als Ursache zugeordnet ist. Die Schwere wird über den Common Vulnerability Scoring System (CVSS) Basiswert abgebildet. Beide Größen werden auf einen Wertebereich zwischen 0 und 1 normiert. Der Danger Score ergibt sich daraus wie folgt:

$$\text{Danger Score} = F_r \cdot S_v \cdot 100$$

$$F_r = \frac{\text{Anzahl}(\text{CWE}) - \text{Anzahl}_{\text{min}}}{\text{Anzahl}_{\text{max}} - \text{Anzahl}_{\text{min}}}$$

$$S_v = \frac{\overline{\text{CVSS}}(\text{CWE}) - \text{CVSS}_{\text{min}}}{\text{CVSS}_{\text{max}} - \text{CVSS}_{\text{min}}}$$

Dabei bezeichnet F_r die normalisierte Häufigkeit einer Schwachstelle und S_v die normalisierte durchschnittliche CVSS Schwere [5]. $\text{Anzahl}(\text{CWE})$ ist die Anzahl der CVE Zuordnungen, $\overline{\text{CVSS}}(\text{CWE})$ der durchschnittliche CVSS Basiswert. Die Indizes min und max bezeichnen jeweils den kleinsten und größten Wert im Datensatz. F_r und S_v liegen damit jeweils zwischen 0 und 1. Schwachstellen, die selten auftreten oder bei Ausnutzung geringe Auswirkungen haben, erhalten einen niedrigen Score. Häufige Schwachstellen mit schwerwiegenden Folgen werden entsprechend höher bewertet.

2.2 Begriff der Speichersicherheit

Ausgehend von der CWE Bewertung stellt Speichersicherheit ein zentrales Sicherheitsproblem dar. Damit ist jedoch noch nicht geklärt, was unter Speichersicherheit, im Englischen Memory Safety, konkret zu verstehen ist. Der Begriff wird häufig verwendet, eine einheitliche Definition ist aber weniger selbstverständlich, als es zunächst wirkt.

Grundsätzlich lässt sich zwischen der Speichersicherheit einer Programmausführung, eines Programms und einer Programmiersprache unterscheiden [7]. Eine Programmausführung gilt als speichersicher, wenn während ihres Ablaufs kein Speicherzugriffsfehler auftritt. Ein Programm gilt als speichersicher, wenn alle möglichen Ausführungen speichersicher sind. Entsprechend gilt eine Programmiersprache als speichersicher, wenn jedes in ihr erzeugbare Programm speichersicher ist.

Eine verbreitete Definition orientiert sich an einer Liste typischer Speicherzugriffsfehler [7], [8]. Solche Fehler zählen zu den ältesten und persistierendsten Sicherheitsproblemen in Software, die in Sprachen wie C oder C++ geschrieben ist. Fehlt Speichersicherheit, können Angreifer das Verhalten von Programmen verändern oder die Kontrolle über den Programmablauf übernehmen. Trotz zahlreicher Schutzmechanismen bleiben Speicherfehler bis heute eine ernsthafte Bedrohung. Tabelle 1 fasst fünf zentrale Speicherzugriffsfehler zusammen. Tritt mindestens einer dieser Fehler in einer möglichen Ausführung auf, ist das Programm nicht speichersicher.

Kurz gefasst lässt sich Speichersicherheit auch so erklären: Ein Programm ist speichersicher, solange es keinen undefinierten Speicher verwendet [7]. Undefinierter Speicher ist Speicher, der nie alloziert wurde oder dessen Lebensdauer bereits beendet ist. Explizit alloziert wird Speicher auf dem Heap, auf dem Stack oder im statischen Datenbereich. Die genannten Speicherzugriffsfehler lassen sich als unzulässige Zugriffe auf undefinierten Speicher interpretieren.

Speicherzugriffsfehler	Zugriffsart	Beschreibung
Out of Bounds Write	Schreiben	Schreibzugriff außerhalb eines gültigen Puffers oder Arrays
Out of Bounds Read	Lesen	Lesezugriff außerhalb eines gültigen Puffers oder Arrays
Use After Free	Lesen oder Schreiben	Zugriff auf Speicher, der bereits freigegeben wurde
Use of Uninitialized Memory	Lesen oder Schreiben	Zugriff auf reservierten, aber noch nicht initialisierten Speicher
Double Free	Freigabe	Erneute Freigabe eines bereits freigegebenen Speicherbereichs

Tabelle 1 : Fünf zentrale Speicherzugriffsfehler [8]

2.3 Use After Free

Use After Free bezeichnet einen Speicherzugriffsfehler, bei dem auf Speicher zugegriffen wird, der bereits freigegeben wurde. Der Fehler tritt ausschließlich im Zusammenhang mit dynamischer Speicherverwaltung auf. Betroffen ist damit Speicher, der zur Laufzeit auf dem Heap alloziert wurde.

Im Gegensatz zum Stack, auf dem lokale Variablen und Funktionsparameter automatisch verwaltet werden, dient der Heap für Speicher, dessen Lebensdauer über den aktuellen Funktionsaufruf hinausgehen soll. Allokation und Freigabe erfolgen in C explizit über Funktionen wie `malloc()` und `free()`. Der Heap ist ein großer Speicherbereich, aus dem der Laufzeitallocator bei Bedarf einzelne Blöcke bereitstellt. Freigegebene Blöcke können bei späteren Allokationen erneut vergeben werden. Diese Flexibilität ermöglicht effiziente Speichernutzung, birgt aber das Risiko, dass Zeiger auf bereits freigegebenen Speicher weiterhin existieren. Das folgende C Beispiel veranschaulicht den Fehler, da C zusammen mit C++ zu den verbreitetsten Sprachen mit manueller Speicherverwaltung zählt.

Nach der CWE Definition greift ein Use After Free vor, wenn ein Programm Speicher erneut verwendet oder referenziert, nachdem dieser freigegeben wurde [9]. Zu einem späteren Zeitpunkt kann derselbe Speicherbereich erneut alloziert. Der ursprüngliche Zeiger verweist dabei weiterhin auf eine Adresse innerhalb der neuen Allokation. Zugriffe über den ursprünglichen Zeiger sind ungültig, da der Speicherbereich faktisch dem Code gehört, der über den neuen Zeiger arbeitet. Ein Zeiger, der nach einer Freigabe nicht mehr auf gültigen Speicher verweist, wird als Dangling Pointer bezeichnet. Typischerweise durchläuft ein Use After Free vier Phasen, die in Abbildung 2 zusammengefasst sind:

1. **Allocate:** Speicher wird auf dem Heap alloziert.
2. **Free:** Der Speicher wird freigegeben. Bestehende Zeiger verweisen danach auf ungültigen Speicher und werden zu Dangling Pointern.
3. **Allocate:** Derselbe Speicherbereich kann erneut alloziert werden.
4. **Use:** Ein Zugriff über den Dangling Pointer liest oder schreibt fälschlich den neu zugewiesenen Inhalt.

Quelltext 1 zeigt ein minimales Beispiel. In Zeile 5 wird über `malloc()` Speicher für einen Integer alloziert und `buffer_ptr1` gesetzt. Zeile 6 schreibt den Wert 21 in diesen Block. Zeile 8 erzeugt mit `buffer_ptr2` einen zweiten Zeiger auf dieselbe Adresse, ohne neuen Speicher zu reservieren. Mit `free(buffer_ptr1)` in Zeile 10 wird der Block freigegeben. Damit endet die Gültigkeit der Allokation. Beide Zeiger enthalten weiterhin die bisherige Adresse und sind ab diesem Punkt Dangling Pointer. Entspricht dies der Free Phase aus Abbildung 2.

Zeile 12 alloziert erneut Speicher für einen Integer, was der zweiten Allocate Phase entspricht. Je nach Allocator und Heap Zustand kann `buffer_ptr3` dieselbe Adresse erhalten wie zuvor `buffer_ptr1`. Zeile 13 überschreibt den Inhalt mit 42.

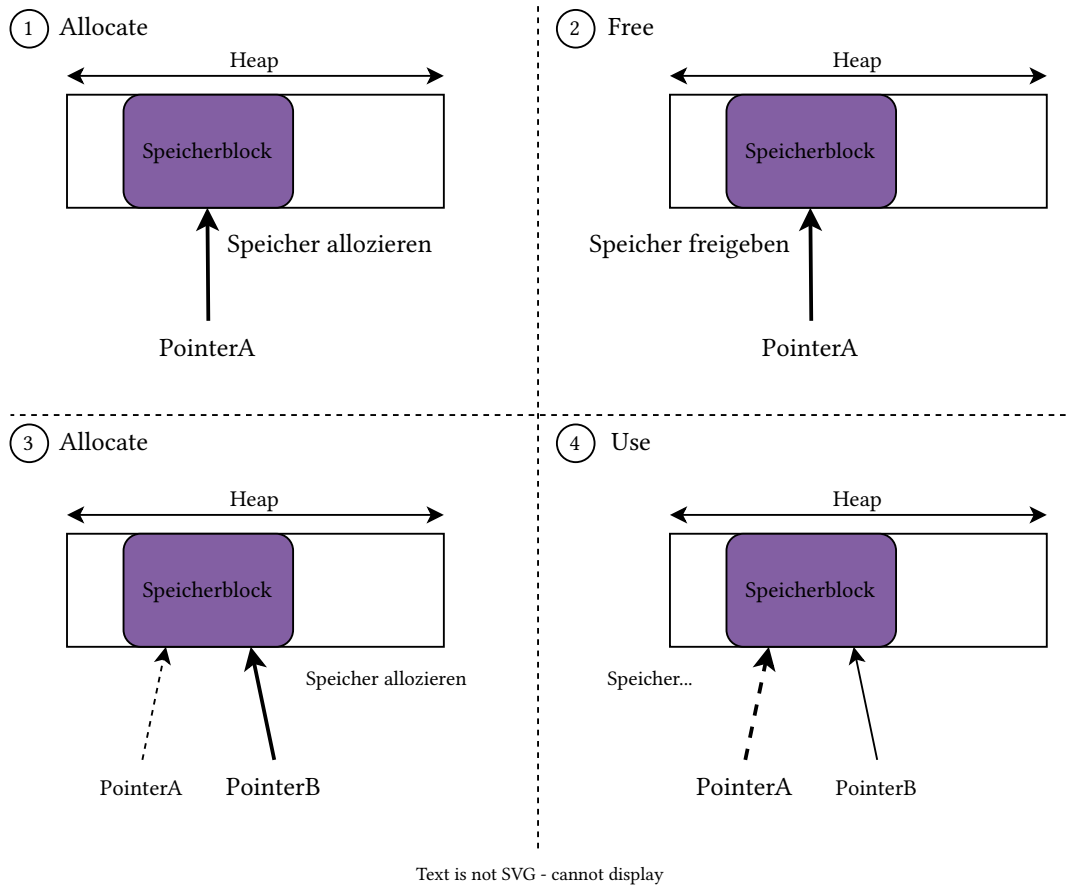


Abbildung 2 : Vierphasiger Ablauf eines Use After Free Fehlers auf dem Heap [9]

Zeile 15 dereferenziert `buffer_ptr2` und gibt den Wert aus. Der Zugriff erfolgt über einen Dangling Pointer und entspricht der Use Phase. Statt des ursprünglichen Werts 21 wird der Wert 42 ausgegeben, der über `buffer_ptr3` geschrieben wurde. Der Fehler entsteht, weil zwei Zeiger auf denselben Speicher verwiesen, die Freigabe aber nur über einen von ihnen durchgeführt wurde.

Dieses Verhalten ist sicherheitskritisch, wenn über den neuen Zeiger vertrauliche Daten abgelegt werden und der Dangling Pointer unberechtigten Lesezugriff ermöglicht. Derartige Fehler entstehen besonders leicht, wenn mehrere Zeiger auf denselben Heap Block verweisen und die Freigabe nicht für alle Zeiger nachvollziehbar ist.

2.4 Out of Bounds Write

Neben Use After Free zählt Out of Bounds Write zu den häufigsten Speicherzugriffsfehlern aus der Einleitung. Anders als Use After Free betrifft dieser Fehler nicht nur den Heap, sondern tritt besonders häufig auf dem Stack auf. Der Stack dient der Verwaltung von Funktionsaufrufen in Programmen. Bei jedem Aufruf werden lokale Variablen, Funktionsparameter und Rücksprunginformationen in einem last in, first out Verfahren abgelegt. Beim Verlassen einer Funktion wird der zugehörige Stackbereich

```c
#include <stdlib.h>
#include <stdio.h>

int main(int argc, char **argv) {
    int *buffer_ptr1 = malloc(sizeof(int));
    *buffer_ptr1 = 21;

    int *buffer_ptr2 =  buffer_ptr1;

    free(buffer_ptr1);

    int *buffer_ptr3 = malloc(sizeof(int));
    *buffer_ptr3 = 42;

    printf("%d", *buffer_ptr2);
}

```

Quelltext 1 : Minimales Use After Free Beispiel in C

wieder freigegeben. Im Gegensatz zum Heap erfolgt diese Verwaltung automatisch durch den Compiler und die Laufzeitumgebung. Typischerweise wächst der Stack dabei in Richtung abnehmender Speicheradressen. Welche Variablen an welcher Adresse liegen, hängt vom Compiler und der Zielplattform ab. Gerade diese feste Nachbarschaft von Daten auf dem Stack macht Out of Bounds Write besonders gefährlich.

Nach der CWE Definition liegt ein Out of Bounds Write vor, wenn Daten hinter das Ende oder vor den Anfang eines vorgesehenen Puffers geschrieben werden [10]. Schreibt ein Programm außerhalb der gültigen Grenzen eines Arrays oder Puffers, kann benachbarter Speicher überschrieben werden. In der CWE Terminologie wird der daraus resultierende Schaden häufig als Memory Corruption bezeichnet, also die Beschädigung von Speicher durch ungültige Schreibzugriffe [10]. Abbildung 3 veranschaulicht den Fehler am Beispiel eines Stacklayouts. Dort liegen eine lokale Variable und ein Array unmittelbar hintereinander. Gültige Indizes des Arrays reichen von 0 bis 1. Wird dennoch über Index 2 geschrieben, trifft der Schreibzugriff nicht mehr den Arraybereich, sondern die benachbarte Variable. Der ursprüngliche Wert der Variable wird so stillschweigend überschrieben.

Quelltext 2 zeigt ein minimales C Beispiel. In Zeile 8 wird die Variable `var` mit dem Wert 5 initialisiert. Zeile 9 legt ein Array mit zwei Elementen an. Zeile 11 ruft `write_arr_at_index(array, 2)` auf und übergibt den Index 2. Für ein Array der Länge 2 sind jedoch nur die Indizes 0 und 1 gültig. Die Funktion in Zeile 3 und 4 führt den Schreibzugriff ohne Prüfung des Index aus. Entsprechend der in Abbildung 3 dargestellten Anordnung kann der Zugriff auf `arr[2]` den Speicher der Variable `var`



Abbildung 3 : Out of Bounds Write auf dem Stack [10]

treffen. Beim GNU C Compiler (GCC) liegen lokale Variablen und Arrays auf dem Stack typischerweise in unmittelbarer Nachbarschaft mit absteigenden Adressen. Zeile 13 gibt daher nicht den erwarteten Wert 5 aus, sondern 99.

Auch wenn die Veränderung einer einzelnen Variable harmlos erscheint, sind die Folgen bei sicherheitskritischem Code gravierend. Liegt an der betroffenen Stackposition statt einer einfachen Variable beispielsweise eine Rücksprungadresse oder ein Funktionszeiger, kann ein gezielter Out of Bounds Write die Programmausführung umleiten und die Ausführung von Angreifercode ermöglichen [8].

```c
#include <stdio.h>

void write_arr_at_index(int *arr, int index) {
    arr[index] = 99;
}

int main() {
    int var = 5;
    int array[2] = {2, 2};

    write_arr_at_index(array, 2);

    printf("%d", var);
}

```

Quelltext 2 : Minimales Out of Bounds Write Beispiel in C

2.5 Sicherheitsrelevante Konsequenzen

Die beiden beschriebenen Fehler wirken sich auf mehrere zentrale Sicherheitsziele aus. Neben Integrität und Verfügbarkeit sind vor allem Vertraulichkeit und die Kontrolle über die Programmausführung betroffen [8], [11].

Vertraulichkeit stellt sicher, dass Informationen nur für berechtigte Empfänger zugänglich sind. Use After Free kann dieses Ziel verletzen, wenn über Dangling Pointer in bereits freigegebenen Speicher gelesen wird. Der Speicherbereich kann inzwischen anderen Teilen des Programms zugeordnet sein, etwa einem neuen Heap Block mit vertraulichen Inhalten. In Quelltext 1 liest `buffer_ptr2` einen Wert aus, der eigentlich über `buffer_ptr3` geschrieben wurde. Übertragen auf realen Code entspricht das dem unbefugten Auslesen fremder Daten aus dem Prozessspeicher.

Integrität bedeutet, dass Daten nicht unbefugt verändert werden. Use After Free gefährdet dieses Ziel auf dem Heap, wenn über Dangling Pointer in bereits freigegebenen Speicher geschrieben wird. Out of Bounds Write bedroht die Integrität vor allem auf dem Stack. Wie in Quelltext 2 gezeigt, überschreibt ein ungültiger Index den Wert einer benachbarten Variable. Allgemeiner lassen sich damit benachbarte Speicherbereiche manipulieren, etwa Metadaten, Zeiger oder Kontrollstrukturen.

Neben der Integrität einzelner Datenwerte kann Speicherkorruption die Kontrolle über den Programmablauf selbst gefährden. Out of Bounds Write erlaubt es, Funktionszeiger oder Rücksprungadressen auf dem Stack zu überschreiben. Use After Free kann zusammen mit weiteren Techniken dazu genutzt werden, unerwartete Ausführungspfade zu erzeugen [8]. Die CWE ordnet Out of Bounds Write ausdrücklich der Folge Execute Unauthorized Code or Commands zu [10]. Damit lassen sich Speicherfehler von reinen Datenfehlern zu Angriffen eskalieren, die die ursprüngliche Logik des Programms vollständig unterlaufen.

Verfügbarkeit beschreibt, ob ein System oder Dienst für berechtigte Nutzer erreichbar bleibt. Speicherfehler können diese Verfügbarkeit durch Abstürze oder instabiles Verhalten einschränken [11]. Bei Use After Free spielt die interne Verwaltung des Heaps eine Rolle. Speicherallocator wie die Implementierung von `malloc` in der GNU C Library führen nach einer Freigabe häufig eine Chunk Konsolidierung durch, bei der benachbarte freie Blöcke zusammengeführt werden. Greift ein Programm danach über einen Dangling Pointer erneut auf den freigegebenen Bereich zu, können die internen Verwaltungsinformationen des Heaps nicht mehr gültig sein. Der Prozess bricht in solchen Fällen oft ab. Out of Bounds Write kann die Verfügbarkeit ebenfalls beeinträchtigen. Wird etwa eine Rücksprungadresse auf dem Stack überschrieben, kehrt die Ausführung nicht mehr kontrolliert in den aufrufenden Code zurück. Auch ungültige Zugriffe auf nicht vorgesehene Speicherbereiche können den Prozess beenden. Für netzwerk- oder serverbasierte Anwendungen bedeutet ein solcher Absturz unmittelbar den Ausfall des betroffenen Dienstes.

2.6 Lösungsansätze

Die gezeigten Beispiele verdeutlichen, warum Speicherfehler in C und C++ ein dauerhaftes Sicherheitsproblem darstellen. Gegen Speicherfehler haben sich in der Praxis im Wesentlichen drei Ansätze etabliert. Sie unterscheiden sich darin, ob Speicher zur Laufzeit automatisch verwaltet, zur Laufzeit überwacht oder bereits zur Compilezeit geprüft.

Der verbreitetste Ansatz ist der Einsatz speichersicherer Programmiersprachen mit automatischer Speicherverwaltung. Dabei übernimmt eine Laufzeitumgebung die Verwaltung des Heaps. Ein Garbage Collector ermittelt zur Laufzeit, welche Speicherobjekte noch erreichbar sind, und gibt nicht mehr benötigten Speicher automatisch frei. Entwickler müssen in solchen Sprachen typischerweise weder `malloc()` noch `free()` aufrufen und können damit Use After Free oder Double Free in der vorliegenden Form vermeiden. Sprachen wie Java, Python oder JavaScript setzen diesen Ansatz unterschiedlich um, gelten aber gemeinsam als speichersicherer als C oder C++. Häufig werden zusätzlich Laufzeitprüfungen für Arraygrenzen eingesetzt, um Out of Bounds Zugriffe abzufangen. Der Preis dafür ist ein zusätzlicher Laufzeitaufwand, der vor allem durch den Garbage Collector entsteht. In performancekritischen Systemen kann dieser Overhead spürbar sein. Für bestehende C Codebasen ist ein vollständiger Wechsel auf eine andere Sprache zudem oft mit erheblichem Migrationsaufwand verbunden [12].

Ein zweiter Ansatz zielt auf bestehenden C oder C++ Code. Dabei wird das Programm beim Kompilieren oder zur Laufzeit so erweitert, dass ungültige Speicherzugriffe erkannt werden, bevor sie unkontrolliert eskalieren. AddressSanitizer (ASan) ist ein verbreitetes Beispiel für diese Kategorie. ASan fügt zusätzliche Prüfungen ein und nutzt sogenanntes Shadow Memory, um nachzuvollziehen, welche Speicheradressen gültig sind. Das Werkzeug findet unter anderem Out of Bounds Zugriffe und Use After Free während der Programmausführung. Laut den Autoren entsteht dabei im Mittel eine Verlangsamung von etwa 73 Prozent [13]. Valgrind verfolgt einen ähnlichen Zweck, arbeitet aber durch dynamische Instrumentierung zur Laufzeit und ist deutlich langsamer. Es eignet sich vor allem für Tests und Fehlersuche in der Entwicklung, nicht für den dauerhaften Produktionseinsatz. Entscheidend ist, dass diese Werkzeuge Fehler erst zur Laufzeit sichtbar machen. Sie verhindern Speicherfehler nicht vollständig und decken nicht alle Fehlerklassen zuverlässig ab [8]. Auch Rust führt bei einigen Prüfungen, etwa Grenzchecks bei Arrayzugriffen, zur Laufzeit einen kontrollierten Abbruch aus, statt undefiniertes Verhalten zu erzeugen. Die zentrale Speichersicherheit wird in Rust jedoch nicht über Instrumentierung, sondern über Compilezeitprüfungen erreicht.

Der dritte Ansatz verschiebt die Erkennung von Speicherfehlern in den Compiler. Potenziell unsichere Programme werden gar nicht erst gebaut, wenn sie gegen die Regeln der Sprache verstoßen. Damit lassen sich Use After Free, Dangling Pointer und viele Out of Bounds Zugriffe bereits vor der Ausführung ausschließen. Im Idealfall entsteht dafür kein dauerhafter Garbage Collection Overhead, weil die Speicherverwaltung statisch geprüft wird statt vollständig zur Laufzeit nachverfolgt zu werden. Dieser Ansatz

erfordert jedoch strengere Sprachregeln und ein Umdenken bei der Programmierung. Rust verfolgt genau diese Strategie mit Konzepten wie Ownership und Borrowing. Im folgenden Kapitel wird dargestellt, wie Rust Speicherfehler zur Compilezeit verhindert und damit einen alternativen Weg zu Garbage Collection und nachträglicher Instrumentierung eröffnet.

3 Rust's Approach

3.1 Slide 19: Why use Rust?

- Rust verspricht die Lösung für die Kombination zweier bislang nicht kombinierbarer Programmier Herausforderungen zu sein
- Volle Hardware und Speicher Kontrolle ohne auf Sicherheit und Entwicklererlebnis zu verzichten
- Rust bietet neben dem herzstück dem Rust Compiler auch viele andere Entwicklungswerkzeuge wie Formatter, Language Server und Paket Manager an
- Scheinbar sind das auch keine leeren Versprechen
- Zumindest wenn man den stackoverflow statistiken vertraut
- Dort ist Rust seit Jahren die beliebteste aufstrebende Programmiersprache
- Über 72% der Entwickler, die angefangen haben Rust zu verwenden, verwenden es auch über die nächsten Jahre hinweg weiter
- aber was muss diese Programmiersprache jetzt bieten, damit die vorgestellten Speicher Fehler sicher erkannt oder abgefangen werden?
- Das schauen wir uns jetzt Stück für Stück an

3.2 Slide 20: Ownership Rules

- Menge an Regeln, die dafür sorgen, dass der Speicher, den ein Programm nutzt sicher verwaltet wird
- Diese Regeln gelten zu Compile Zeit, es bedarf also keinen Garbage Collector
- Die Ownership regelt die Besitzrechte von Daten im Speicher
- dafür gelten die folgende Regeln:
  1. Jeder Wert in Rust hat einen Besitzer einen Owner, in diesem Beispiel ist buffer_owner jetzt der aktuelle Besitzer
  2. Es kann immer nur einen Besitzer gleichzeitg geben, ab dem Punkte wo another_buffer_owner der Besitzer wird ist buffer_owner kein Besitzer mehr
  3. Wenn der Besitzer out-of-scope also nicht mehr sichtbar ist, dann wird der Wert verworfen, out of scope kann in Rust durch verschiedene Konstrukte passieren:
     - Gültigkeitsbereich wird verlassen da Block mit geschweiften Klammern zu Ende ist
     - würde man buffer_owner einen neuen Wert mittels Box::new(...) zuweisen würde wir der vorherige Wert auch verworfen
     - würde man buffer_owner als Parameter in eine Funktion geben, dann verlässt dieser auch den aktuellen Scope

- immer wenn ein Besitzer den Gültigkeitsbereich verlässt, dann wird durch Rust automatisch der drop() Aufruf eingefügt, um den Wert zu verwerfen
- diese Regeln bringen einen enormen Vorteil für die Entwicklung:
  - der Entwickler muss nicht manuell Speicher wieder freigeben, die Instruktionen dafür fügt der Rust Compiler von alleine ein
  - Somit kann das Freigeben von Speicher auch nicht vergessen werden

3.3 Slide 21: Transfer of Ownership

- Regel 2, also das es nur einen Besitzer gleichzeitig geben darf, wird auf eine bestimmte Art und Weise realisiert
- würde es sich nicht um Speicher handeln, der auf dem Heap alloziert wurde sondern auf dem Stack, dann wäre der Sachverhalt klar
- die Daten würden einfach kopiert werden, Bsp: let var1 = 42; let var2 = var1; danach existieren zwei Variablen mit dem gleichen Wert
- aber ausgehend von diesem Beispiel: let buffer_owner = Box::new(42); let another_buffer_owner = buffer_owner;
- was genau passiert mit s1, wenn s1 die Besitzrechte abgeben muss?
- es kann sich bei dynamischen Größen auf dem Heap schnell im große Datenmenge handeln, wodurch viel Speicher Overhead und Performance Einbuse durch das Kopieren passieren würde
- daher is die Frage? Ist Kopieren wirklich die beste Idee?

3.4 Slide 22: Transfer Methods

- prinizipiell gibt es drei verschiedene Möglichkeiten wie die Besitzübergabe von s1 nach s2 passieren könnte:
  1. Deep Copy: Dabei wird der gesamte Inhalt der Speicher Einheit, die auf dem Heap allooziert ist kopiert und zusätzlich auch noch der darauf zeigende Pointer kopiert
  2. Shallow Copy: Hier wirde nur der Pointer kopiert, dieser zeigt aber auf die selbe Speicheradresse der Daten im Heap, diese müssen also nicht kopiert werden
  3. Move: Hierbei wird zunächst auch der Pointer kopiert, dann aber im Gegensatz zum Shallow Copy der Zeiger s1 gelöscht, sodass nach einem Move nach wie vor nur ein Zeiger auf eine Speicherzelle zeigt
- das Problem beider Copy Varianten ist, dass nach dem Kopieren auf jeden Fall mehr Zeiger vorhanden sind als davor, das bedeutet automatisch ein größerer Aufwand beim freigeben von Speicherzellen
- Shallow Copy hat zusätzlich noch das Problem, dass es extrem anfällig für Dangling Pointer ist, also Zeiger, die auf einen bereits freigegebenen Speicherbereich zeigen

3.5 Slide 23: Move Semantics

- Rust setzt standardmäßig bei der Übergabe einer Ownership IMMER auf Move
- um genau die Probleme, die durch Copy entstehen können zu vermeiden

- Speicher kann dadurch nicht fälschlicherweise doppelt freigegeben werden, da der Speicher immer nur in Verwendung eines einzige Besitzers ist
- Kopieren ist zwar immer noch möglich in Rust, aber nur durch das explizite Aufrufen einer Clone() Funktion

3.6 Slide 24: Borrowing

- was macht man nun aber, wenn man nicht immer die Besitzrechte zu einem Wert überall mit durchreichen möchte
- bspw. bei Funktionsaufrufen könnte es nützlich sein auf einen Wert zuzugreifen ohne sich auch noch um die Ownership zu kümmern
- dafür gibt es Referenzen -> eine Adresse, an der sich ein Pointer befindet, der auf Speicherbereich zeigt
- um eine Referenz anzulegen muss man sich die Besitzrechte dafür leihen, im Code geschieht das über das kaufnmännische UND Zeichen, die Funktion increment bekommt also eine Referenz auf var übergeben
- das Erstellen so einer Referenz nennt man in Rust Borrowing -> Wert verwenden ohne Ownership zu übergeben
- es gibt auch nach dem Erstellen einer Referenz nur einen Owner
- Durch Borrowing ist es also hier im Code Beispiel möglich die Variable var zu inkrementieren ohne die Besitzrechte zu übergeben
- Referenzen sind per default immutable also unveränderbar
- es ist aber möglich auch veränderbare Referenzen anzulegen, das geschieht aber wieder nur explizit
- falls eine Referenz aber als veränderbar markiert ist, kann es abseits von dieser Referenz keine anderen geben
- ganz nach dem Konzept: wenn geschrieben wird, dann gibt es nur diesen einen Writer, falls nur gelesen wird, kann es auch mehrere lesende Referenzen geben
- dieses Konzept verhindert zusätzlich noch eine ganz andere große Fehlerquelle nämlich Data Races, um die soll es hier jetzt aber nicht weiter gehen
- Dadurch das es möglich ist sich die Zugriffsrechte auf Daten auszuleihen, könnte man meinen es sei auch wieder möglich Dangling Reference zu provozieren
- da ohne zusätzliche Absicherung eine Referenz theoretisch ohne zugehörige valide Daten existieren könnte

3.7 Slide 25: Lifetimes and Borrow Checker

- Aber auch hier bietet der Rust Compiler ein Konzept, um diese Dangling References zu Compilezeit vollständig zu verhindern
- dafür gibt es in Rust sogenannte Lifetimes
- Lifetimes geben an für wie lange oder über welchen Programmteil hinweg Variablen gültig sind
- hier veranschaulicht durch die Pfeile, die zeigen in welchem Teil die Referenz und der Wert gültig sind

- Der Wert ist zum zugriffzeitpunkt per println über die Referenz nicht mehr gültig
- das wird durch den Borrow Checker erkannt
- der borrow Checker überprüft die Lebenszeiten und vergleicht diese, um herauszufinden, ob alle Ausleigen valide sind
- da die Lebenszeit des Variable value hier kürzer ist als die der Referenz, meldet der Borrow Checker einen Fehler zur Compilezeit
- dadurch können Dangling References vollständig zur Compilezeit erkannt werden

3.8 Slide 26: Prevention of Use after Free

- so das waren jetzt viele theoretische Konzepte auf einmal
- jetzt wollen wir anschauen wie genau diese in der Sprache Rust zum Einsatz kommen um ein Use After Free zu verhindern
- dafür erstellen wir genau wie im Beispielcode in C eine Variabel, die Speicher für einen Integer auf dem Heap reserviert
- Box ist hier ein Smart Pointer, mit dem man mittels new() einen bestimmten Wert, in unserem Fall einen Integer auf dem Heap gespeichert werden kann
- normalerweise lohnt sich das nur für Typen, die eine variable Größe haben so wie String
- wie wollen aber möglichst nah am C Beispiel bleiben und verwenden daher Box
  1. In der ersten Code Zeile wir dann also nach den Ownership Regeln der Besitzer für den Wert auf dem Heap festgelegt
  2. In C haben wir dann einen zweiten Pointer festgelegt, der auf den gleichen Speicher wie Pointer 1 zeigt
     - Das ist in Rust so direk garnicht möglich, weil in C dafür ein Shallow Copy eingesetzt wird
     - Shallow Copy haben wir aber gesehen gibt es nicht so direkt in Rust
     - Um aber trotzdem Zugriff auf den Wert im Heap zu bekommen erstellen wir eine Referenz
     - buffer_borrow leiht sich also den Zugriff nur aus
  3. Wir rufen explizit drop() auf um buffer_owner zu verwerfen, untypisch in Rust drop() explizit aufzurufen, aber hier einfach für Demo Zwecke
     - Dadurch eendet die Lebenszeit von buffer_owner
  4. Nun versuchen wir den use after free Fehler zu provozieren
     - wir versuchen auf Speicher zuzugreifen, der bereits freigegeben wurde indem wir über buffer_borrow auf die Daten zugrifen wollen und diese printen wollen

3.9 Slide 27: Prevention of Use after Free

- wir bekommen im Gegensatz zu C einen Fehler beim Kompilieren
- es ist also garnicht möglich ein Executable zu erstellen, was so einen Bug beinhaltet
- die Fehlermeldung zeigt genau das was wir gerade detailliert durchgesprochen haben
- und zusätzlich noch warum ein Fehler aufgetreten ist

- der Borrow Checker im Compiler hat nämlich detektiert, dass die Lebenszeit von buffer_borrow länger als die des ZUgehöriger Besitzers buffer_owner ist
- Das erlauben die Rust Lifetime Regeln nicht und daher wir ein Fehler angezeigt
- Zusätzlich erhält man noch den Vorschlag, was man ändern könnte, damit kein Fehler aufritt
- der Compiler schlägt ein Deep Copy mittels clone() Funktion vor und weißt auch direkt auf zusätzlich Ressourcen Kosten hin

3.10 Slide 28: Compiletime Bounds Checking

- zu Beginn hatte ich gezeigt, dass direkt nach dem Use-After-Free Fehler der out-of-bounds write auch eine große Schwachstelle darstellt
- auch dafür bietet Rust eine bzw. um genauer zu sein zwei verschiedene Lösungsansätze an
- die am einfachsten zu erkennenden Out-of-Bounds Fehler sind die, bei denen der verwendete Index zur Compiletime schon bekannt ist
- hier im Beispiel ist beim Kompilieren zu 100% klar, dass der Index der beim Zugriff in Zeile X verwendet wird 2 ist
- Dadurch kann der Rust Compiler der Zugriff über die Array Grenzen hinweg auf jedem Fall zur Compiletime erkennen

Slide 29: Runtime Bounds Checking

- Anders sieht das aus, wenn man versucht, das C Beispiel vom Anfang in Rust zu transformieren
- In diesem Beispiel passiert der Out-Of-Bounds Write nämlich innerhalb eines Funktionsaufrufs
- die Funktion nimmt einen zur Laufzeit übergebenen Index an mit dem dann auf das Array zugegriffen wird
- zur Compilezeit sicher zu beweisen, dass so ein Fehler auftritt ist quasi unmöglich
- Daher muss ein anderes Konzept her mit dem solche Fehler keine größeren Sicherheitslücken oder Schaden anrichten können
- Rust kann solche out-of-bounds Zugriff nämlich auch feststellen aber nur zur Laufzeit
- dafür wird für jeden Index Zugriff zur Laufzeit geprüft, ob dieser innerhalb der Array Grenzen liegt
- falls dies nicht der Fall ist wird eine Panic Hook ausgeführt
- das erzeugt natürlich einen Runtime Overhead
- dieser ist aber sehr gering, da im Allgemeinen nur ein Vergleich und ein bedingter Sprung für die Überprüfung benötigt werden
- die dafür notwendigen Instruktionen generiert der Rust Compiler immer ohne dass sich der Entwickler das explizit erzwingen muss
- sobald ein out-of-bounds zugriff festgestellt wird gibt es zwei möglich panic handler je nach Einstellung

  - bei der Einstellung unwind wird der Stack abgebaut und nach Fehlerbehandlung ist eine weitere Programmausführung möglich
  - bei der Einstellung abort wird der aktuelle Prozess abgebrochen
- aber egal welche Einstellung gewählt wird, das Verhalten bei einem out-of-bounds zugriff versuch ist sicher dadurch, dass vor dem tatsächlichen Zugriff kontrolliert der Panic Handler aufgerufen wird, während in C das Verhalten in so einem Fall völlig undefiniert bleibt

4 Fazit

- jetzt haben wir an einigen wenigen Beispielen gesehen wie Rusts Konzepte dafür sorgen können Memory safen coden zu entwickeln
- aber sind diese Konzepte wirklich allgemein gültig und ist jedes Programm, welches mit Rust entwickelt wird zu 100% Memory safe?
- und ja natürlich löst Rust nicht alle Memory Safety Probleme, was vor allem daran liegt, dass es in Rust auch möglich ist explizit unsicheren Code zu schreiben, um Beispiel mit perse unsicherem C Code zu interagieren oder Low-Level-Code für Hardware Interaktion zu entwickeln
- außerdem sind die Programmier Konzepte nicht unbedingt schnell für Entwickler zu erlernen, was die Entwicklungs anfangs verlangsamen kann
- dennoch wurde von Forschern bewiesen, dass das Ownership Modell in Rust erhebliche Memory Safety Probleme durch das Design unterbindet
- und auch in der Praxis wirkt die Verwendung von Rust
- Google berichtete in ihrem security blog 2024, dass nach Einführung von Rust in Android die Anzahl memory-safety Schwachstellen von 76% auf 24% der Sicherheitslücken zurückging
- damit möchte ich den Vortrag beenden und hoffe ich habe Ihnen ein paar Gründe mehr gegeben Rust einmal auszuprobieren
- ich bin jetzt gerne bereit noch alle Ihre Fragen zu beantworten

Literaturverzeichnis

[1] "CWE - Common Weakness Enumeration." Accessed: May 30, 2026. [Online]. Available: https://cwe.mitre.org/index.html

[2] "CWE - 2025 CWE Top 10 KEV Weaknesses." Accessed: May 30, 2026. [Online]. Available: https://cwe.mitre.org/top25/archive/2025/2025_kev_list.html

[3] "CWE - 2024 CWE Top 10 KEV Weaknesses." Accessed: May 30, 2026. [Online]. Available: https://cwe.mitre.org/top25/archive/2024/2024_kev_list.html

[4] D. E. Mann and S. M. Christey, "Towards a Common Enumeration of Vulnerabilities."

[5] "CWE - 2025 CWE Top 25 Most Dangerous Software Weaknesses." Accessed: May 30, 2026. [Online]. Available: https://cwe.mitre.org/top25/archive/2025/2025_methodology.html

[6] "CWE - 2025 CWE Top 10 KEV List Insights." Accessed: May 30, 2026. [Online]. Available: https://cwe.mitre.org/top25/archive/2025/2025_kev_insights.html

[7] M. Hicks, "What is memory safety? - The PL Enthusiast." Accessed: May 30, 2026. [Online]. Available: http://www.pl-enthusiast.net/2014/07/21/memory-safety/

[8] L. Szekeres, M. Payer, T. Wei, and D. Song, "SoK: Eternal War in Memory," in *2013 IEEE Symposium on Security and Privacy*, May 2013, pp. 48–62. doi: 10.1109/SP.2013.13.

[9] "CWE - CWE-416: Use After Free (4.20)." Accessed: May 30, 2026. [Online]. Available: https://cwe.mitre.org/data/definitions/416.html

[10] "CWE - CWE-787: Out-of-bounds Write (4.20)." Accessed: May 30, 2026. [Online]. Available: https://cwe.mitre.org/data/definitions/787.html

[11] T. Baker, "Memory Safety: An Explainer." Accessed: May 30, 2026. [Online]. Available: https://cset.georgetown.edu/article/memory-safety-an-explainer/

[12] J. V. S.-A. team and A. R.-S. Foundations, "Eliminating Memory Safety Vulnerabilities at the Source." Accessed: May 30, 2026. [Online]. Available: https://security.googleblog.com/2024/09/eliminating-memory-safety-vulnerabilities-Android.html

[13] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov, "AddressSanitizer: A Fast Address Sanity Checker," 2012, pp. 309–318. Accessed: May 30, 2026.

[Online]. Available: https://www.usenix.org/conference/atc12/technical-sessions/presentation/serebryany